Agentic Visual Inspector

A vision system that doesn't just detect a defect — it investigates it, reasons about the cause against the line's recent past, and recommends an action a supervisor can read and challenge.

OpenCV AI Competition 2026, powered by AWS.

---

1. Problem and users

Factory and plant QA lines miss subtle defects because inspectors fatigue, and they raise false alarms because a detector cannot weigh context. An isolated blemish and the third identical blemish in an hour are not the same event, but a per-frame detector reports them identically. Detection is now a commodity; **the decision is where the money and the safety are**. That gap — between a box on a frame and a defensible action — is what this entry addresses.

Users:

- **Line operators**, who need a yes/no "stop or continue" that is right, fast, and local.
- **QA supervisors**, who need to trust the call: they must be able to read *why* it was made and disagree with it.
- **Plant engineers**, who need the line's recent history folded into each decision, because trend is the signal a single frame cannot carry.

The system is designed so the perception half can run at the edge with no cloud on the critical path, and the reasoning half can run in AWS. That split is deliberate: a stopped line cannot wait on a network round-trip to *see*, but it can wait for the *judgement*.

2. Architecture

See `ARCHITECTURE.svg`. Three zones:

1. **Edge / line** — camera → **OpenCV 5** `dnn` running a YOLOv10 ONNX detector on CPU → detection + rendered overlay → the **line history** store.
2. **AWS cloud** — the **reasoning agent**, a bounded tool loop, calls Amazon **Bedrock** (Converse API) for the judgement and writes a **decision record**.
3. **Operator** — a dashboard showing the frame, the detection, the decision, and the reasoning trace; a human can read and challenge it; `critical=true` escalates to an alert.

Perception is local and dependency-light on purpose. The whole perception stage is `opencv-python` + an ONNX file — no service, no GPU, no vendor SDK — which is exactly what makes edge deployment credible.

3. OpenCV 5 implementation

The perception stage uses OpenCV 5's DNN module:

```
 blob = cv2.dnn.blobFromImage(frame, 1/255.0, (640, 640), swapRB=True, crop=False)
net.setInput(blob)
out = net.forward()          # YOLOv10: (1, 300, 6) → x1, y1, x2, y2, score, class
```

`opencv_inspector/detector.py` wraps this into a reusable `Detector` with a persistent net (one `readNetFromONNX`, reused per frame — loading per frame would dominate latency). YOLOv10's export is **NMS-free**, so no cross-class suppression step is needed; that, plus a 640×640 input, is what holds CPU latency to the range measured below. COCO classes are filtered to those meaningful on a plant floor (`RELEVANT`), so the inspector reports things that matter rather than every chair in a corner.

Measured: OpenCV **5.0.0**, **5 detections in ~110–135 ms** on a laptop CPU (bus + 4 people in the bundled sample), no GPU, no CUDA.

4. AWS deployment

The AWS component is the **reasoning inference**. `opencv_inspector/reasoning.py` defines a pluggable backend with an identical contract:

- `LocalBackend` — any OpenAI-compatible endpoint (used for development here; zero cost).
- `BedrockBackend` — Amazon Bedrock **Converse API**, with tool use and region scoping.

Switching is one environment variable:

```
export REASONING_BACKEND=bedrock
export BEDROCK_MODEL_ID=anthropic.claude-3-5-sonnet-20241022-v2:0
export AWS_REGION=us-east-1
```

Status, stated honestly: the Bedrock path is implemented and wired, but it has **not been run against a live AWS account** — there is no AWS account available on the development machine. What is verified is the *contract*: the local backend exercises the identical loop, prompt and output shape, so the Bedrock swap changes the transport, not the system. The deployment intent for further scale-out is COOL (Cloud-Optimized OpenCV) on Graviton: the perception calls are host-agnostic Arm-clean C calls, so moving perception onto the Arm path at scale is a deploy target rather than a rewrite. This is the honest state; it is not claimed as a working AWS deployment.

5. The agentic part

A single prompt is not agency, so the reasoning stage is a **bounded tool loop** rather than one call:

1. Receive the detection and the retrieved prior defects for that station.
2. If the history looks thin, **call** `get_line_history(station, limit)` to widen the window — at most `MAX_TOOL_CALLS = 2` times, so the loop can never spin.
3. Return a strict JSON decision: `critical`, `severity`, `matched_prior`, `pattern`, `action`, `reasoning`.

`MATCHED_PRIOR` uses a retrieval that ranks **class match first, recency second** — the two signals a human supervisor actually uses when they say "this looks like the one from an hour ago". The system prompt forbids inventing history; every fact the agent reasons over was passed to it or fetched from the store, and the test below asserts that constraint.

Verified live (sample frame): the detector's top detection (bus, 0.919) was investigated against the station's history; the agent judged it **HIGH** severity, matched it to a prior, described the pattern, and recommended a halt — with a readable reasoning trace and the tool/step trace logged alongside.

6. Evaluation

Check	Method	Result
Perception works	`detect.py` on the bundled sample	**5 detections (bus + 4 person)** in ~110–135 ms, CPU
Baseline regression	`tests/test_pipeline.py::test_perception_finds_objects`	`>=4` detections, bus present, latency sane — **pass**
Retrieval logic	`test_history_ranks_matching_class_first`	matching class outranks others; the more recent of two matches wins — **pass**
Reasoner contract	`test_reasoner_contract_and_no_invented_history`	all six keys present, typed correctly — **pass**
No fabricated history	same test, with the guard	**pass** — the guard was separately shown to *fire* on an invented prior (`D-999`), so it is not vacuous
Web endpoint	`POST /api/inspect` end-to-end	detections + decision + annotated frame returned — **pass**

The web endpoint is live at `python app.py` → `http://127.0.0.1:5092`; `GET /api/health`, `/api/history`, and `POST /api/inspect` are all exercised by the demo video, which is unedited footage of the running system.

Latency budget (measured, laptop CPU): OpenCV DNN forward pass ~110–135 ms; the reasoning call is the only network-bound step and is off the perception critical path by design.

7. Limitations

- **No labelled evaluation set.** The numbers above are functional and latency measurements, not precision/recall over an annotated defect corpus. A rigorous version needs a labelled sample of the target defects. This is the largest gap and is stated rather than hidden.
- **The AWS path is unexercised against a live account** (§4).
- **The detector is a general COCO model.** It finds people/vehicles — a stand-in for a plant-floor defect model, which would need fine-tuning on the real defect classes. The pipeline is model-agnostic; only the weights change.
- **The reasoning quality depends on the model.** A weak backend will produce weak judgements; the loop constrains the *shape* and the *grounding*, not the wisdom.
- **The line history is per-process SQLite** — right for one line/edge node, not a fleet. A fleet deployment moves this to a shared store.

8. Responsible-use considerations

- **Human control stays in the loop.** The system recommends and escalates; it does not actuate machinery. `action` is advice for a person.
- **The decision is challengeable by construction.** The reasoning trace and the retrieved history are shown next to the verdict, so an operator can see the evidence and override it rather than obey a score.
- **No fabricated evidence.** The agent is forbidden from claiming history it was not given, and a test enforces it. In a safety-adjacent tool a confident invented past is worse than "I don't know", so this constraint is treated as load-bearing, not cosmetic.
- **Camera privacy.** A plant-floor camera captures people. The `RELEVANT` class filter and the local-first design keep frames on the edge by default; only a detection and its context, not raw video, need to leave the machine for the reasoning step.

Repository layout

```
 app.py                    web endpoint (judge demo)
detect.py                  standalone perception check
agentic_reason.py          original reasoning demo
opencv_inspector/
  detector.py              OpenCV 5 DNN + ONNX, render overlay
  reasoning.py             bounded tool loop; Local + Bedrock backends
  line_history.py          retrieval over the line's recent defects
tests/test_pipeline.py     perception, retrieval, contract + grounding guard
ARCHITECTURE.svg           the diagram above
BUILD.md                   exact build / deploy / test
requirements.txt           pinned to the verified versions
```